

XM · CLASE 4



Soluciones de Analítica con Databricks

Clase 4: Ingesta y republicaciones: Auto Loader y bronce idempotente

Docente: Manuela Larrea Gómez

Agenda de la clase 4



Del upload manual a un proceso que vigila la carpeta y no duplica

01

45 MIN

Teoría: el dato llega tarde y se corrige

Republicaciones en el mercado · Auto Loader · esquema, evolución y rescate · idempotencia · metadatos · vista vigente · Lakeflow Connect y CDC

02

10 MIN

Sector: AMI, Octopus Energy, Shell, SSE

Qué cambia con medición de 15 minutos · cómo lo resolvieron otros, con cifras y fuentes

03

65 MIN

Lab 4: bronce con Auto Loader

23 publicaciones como 23 archivos · correr dos veces · llega raw_v2 con columna renombrada y días repetidos · vista vigente · ADR-002

04

10 MIN

Cierre

Hito 2 cerrado · tag s04 · así sería en Azure · próxima clase: calidad y pipelines declarativos

INGESTA

El dato llega tarde y se corrige

Cualquier ingesta que asuma lo contrario funciona en la demo y falla en producción

El problema, en nuestros propios datos

Lo que midieron en la clase 2 (lab02, hecho 2)

23

fechas de publicación para 206 días de dato

5 · 68 · 205

días de retraso entre Fecha y FechaPublicacion (mín · mediana · máx)

200

días publicados de golpe el 25 de julio

0

republicaciones en el archivo de hoy. Van a llegar



Llega tarde

Un job que corre 'cada día a las 6' no encuentra el día de ayer: encuentra lo que XM publicó ayer, que puede ser de hace dos meses. La ingesta se dispara por llegada, no por calendario.



Se corrige

El mismo día de dato vuelve a publicarse con valores distintos meses después. No es un error de XM: es la liquidación del mercado. Bronce tiene que guardar las dos versiones.

Hoy: un upload manual de un Excel. Al final de la clase: un proceso que vigila landing, carga lo nuevo y no duplica.

Republicaciones: así funciona la liquidación del mercado

Un mismo día de dato tiene varias versiones a lo largo de más de un año

Colombia · XM (ASIC)

Versiones de liquidación y ajustes

La liquidación de un mes se publica en versiones sucesivas (preliminar, TX1, TX2, TX3 ... definitiva) a medida que llegan medidas definitivas y reclamaciones. Ejemplo público: el ajuste #1 de la liquidación de marzo de 2025 se publicó el 23 de mayo de 2025 en versión TX3.

Reino Unido · Elexon (BSC)

Corrida	Cuándo
II · informativa	1 semana
SF · inicial (primer cobro)	1 mes
R1 · R2 · R3 · reconciliaciones	2 · 5 · 7 meses
RF · final (97 % con medida real)	14 meses
DF · disputas	28 meses

Conclusión de ingeniería: bronce acumula todas las versiones. 'Vigente' es una decisión que se toma al leer, no al escribir.

Pregunta para la sala: ¿cuántas versiones publica hoy XM por mes y hasta cuántos meses después? Corrijan la tabla si hace falta.

Fuente: xm.com.co/noticias/7884-publicacion-ajuste-1-de-marzo-de-2025-en-version-tx3 · elexon.co.uk/bsc/glossary/2nd-reconciliation

Tres decisiones antes de escribir código (van al ADR-002)

Son decisiones de arquitectura: quedan escritas, con alternativas descartadas



¿Sobrescribir o acumular?

Acumular. Bronze es append-only. Sobrescribir destruye la evidencia de qué se publicó cuándo; auditoría la va a pedir.

Alternativa descartada: MERGE por llave en bronce



¿Cuál es la llave de versión?

La fecha de publicación. Una fila se identifica por llave de serie + día del dato + FechaPublicacion.

Alternativa descartada: `_ingested_at` como versión



¿Qué significa 'vigente'?

La versión más reciente por serie-día. Lo resuelve una vista (hoy) y plata (clase 6). Bronze guarda; alguien más elige.

Alternativa descartada: borrar la versión anterior

docs/adr/ADR-002-republicaciones.md · estado: aceptada · tres decisiones, tres alternativas descartadas

Auto Loader: 'vigila esta carpeta y cárgame lo nuevo'

Fuente de Structured Streaming con formato cloudFiles. Descubre archivos nuevos de forma incremental y recuerda cuáles ya procesó



Descubre lo nuevo

Lista o recibe notificaciones de archivos nuevos. Escala a millones de archivos por hora sin relistar toda la carpeta.



Checkpoint

Guarda qué archivos procesó. Es el estado del sistema: se conserva, se respalda, no se borra a la ligera.



availableNow

Procesa todo lo pendiente y termina: un job por lotes con garantías de streaming. En producción, un job cada hora o por llegada de archivo.



Formatos

CSV, JSON, Parquet, Avro, ORC, texto, binario. El Excel no: por eso hoy lo convertimos a CSV en landing.

```
bronze.py
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "csv")
  .option("cloudFiles.schemaLocation", chk)
  .option("rescuedDataColumn", "_rescued_data")
  .schema(ESQUEMA) # explícito
  .load(f"{volumen}/landing")
  .transform(normalizar)
  .transform(con_metadatos)
  .writeStream
  .option("checkpointLocation", chk)
  .trigger(availableNow=True)
  .toTable(tabla_bronze)) # append
```

Esquema explícito, evolución y columna de rescate

En bronze declaramos lo que esperamos. La pregunta es qué pasa cuando llega algo distinto

cloudFiles.schemaEvolutionMode	Qué hace cuando llega una columna nueva	Cuándo
addNewColumns (defecto sin esquema)	Agrega la columna al esquema y reinicia el stream una vez	Fuentes que crecen y quieres seguir todo
rescue	No cambia el esquema; lo que no encaja va a <code>_rescued_data</code> (JSON)	Bronze con esquema declarado: nada se pierde, nada entra sin revisar
failOnNewColumns	Falla; alguien mira	Contratos estrictos, fuentes reguladas
none	Ignora columnas nuevas (se pierden si no hay rescate)	Casi nunca

```
-- fila de raw_v2 en bronze, con Valor renombrado a ValorKwh
Valor = NULL
_rescued_data = '{"ValorKwh":"1234567.8","_file_path":"../raw_v2.csv}"'
```

Regla del curso

Esquema explícito + rescatar. Perder datos en silencio es lo único que no se permite. Renombres conocidos se resuelven con un mapa de alias en src.

Idempotencia: correr dos veces = correr una

Si el job se cae a la mitad y lo relanzan, no duplica. Eso no es opcional en una ingesta



Checkpoint + append

Auto Loader procesa cada archivo exactamente una vez gracias al checkpoint. Volver a correr no hace nada. Es lo que usamos en bronce: barato y suficiente.



MERGE por llave

Idempotencia por contenido: si la fila existe, actualiza; si no, inserta. Más flexible, más caro. Es lo que hará plata (clase 6), no bronce.



Lo que rompe la garantía

Borrar el checkpoint sin borrar la tabla: reprocesa todo y duplica. Cambiar la ruta del checkpoint: igual. Escribir a mano en bronce: igual.

```
-- prueba de idempotencia del lab: correr la ingesta dos veces y comparar
SELECT count(*) AS filas, count(DISTINCT _source_file) AS archivos FROM bronce; -- 145.408 * 23, las dos veces
-- reconstruir desde cero (landing es la fuente de verdad): DROP TABLE + borrar checkpoint + correr
```

Tres metadatos que toda tabla bronce lleva

No vienen en el archivo. Responden cualquier pregunta de auditoría: qué archivo, cuándo, qué versión

Columna	De dónde sale	Para qué
<code>_ingested_at</code>	<code>current_timestamp()</code> al escribir	Cuándo entró; frescura; depurar
<code>_source_file</code>	<code>_metadata.file_path</code> (columna oculta de Spark)	Procedencia; reprocesar un archivo
<code>_publication_date</code>	FechaPublicacion del archivo (versión de negocio)	Elegir 'vigente'; historial

```
src/xm_demanda/ingest/bronze.py
def con_metadatos(df):
    return (df
            .withColumn("_ingested_at",
                        F.current_timestamp())
            .withColumn("_source_file",
                        F.col("_metadata.file_path"))
            .withColumn("_publication_date",
                        F.to_date("FechaPublicacion")))
```

Función pura, con test

Recibe un DataFrame, devuelve otro. Se prueba en tests/unit con un DataFrame de tres filas. El notebook solo la llama.

Lo que esto permite en la clase 15

'¿Qué archivo trajo este valor que cambió el pronóstico?' se responde con un WHERE, no con una investigación.

Bronce acumula, la vista elige

'Vigente' = la publicación más reciente por serie-día. Se calcula al consultar; no hay copia

lab04 · paso 7

```
CREATE OR REPLACE VIEW demanda_raw_vigente AS
SELECT * EXCEPT (rn) FROM (
  SELECT *,
    row_number() OVER (
      PARTITION BY Fecha, CodigoVariable, CodigoSICAgente,
        MercadoComercializacion, TipoMercado,
        ClasificacionIndustrial
      ORDER BY _publication_date DESC, _ingested_at DESC
    ) AS rn
  FROM demanda_raw_bronze)
WHERE rn = 1;
```

150.350

filas en bronce tras raw_v2
(145.408 + 4.942)

145.408

filas en la vista vigente: una
por serie-día-variable

Historial

Para los 7 días republicados, bronce tiene dos filas por serie: la de julio y la de agosto. La vista devuelve la de agosto. El historial sigue ahí para auditoría y para medir cuánto cambian las correcciones.

Plata (clase 6) materializa esta lógica con validaciones. Bronce nunca decide qué es verdad; solo recuerda todo.

Cuando la fuente no es un archivo: Lakeflow Connect y CDC

Auto Loader es para carpetas. Bases de datos y SaaS tienen conectores gestionados



Lakeflow Connect

Conectores gestionados para SQL Server, Oracle, PostgreSQL, Salesforce, Workday, ServiceNow... Captura de cambios (CDC) sin escribir el extractor. En XM: la base del SIC.



AUTO CDC en pipelines

Aplica inserts, updates y deletes a una tabla por llave y columna de secuencia (SCD tipo 1 o 2). Es lo que Octopus hace con Change Data Feed: procesar solo lo que cambió.



Cuál usar

Archivos que alguien deja en una carpeta → Auto Loader. Tablas de una base operativa → Connect. Eventos (SCADA, medidores) → Kafka / Event Hubs + Structured Streaming.

Fuente en XM	Mecanismo	Frecuencia
Archivos publicados (demanda, pérdidas, precios)	Auto Loader sobre landing	Por llegada
Base transaccional del SIC / medidas de frontera	Lakeflow Connect (CDC)	Continua o cada 15 min

Fuente: docs.databricks.com/aws/en/ingestion/ · docs.databricks.com/aws/en/dlt/cdc

Sector: lo que viene con medición avanzada (AMI)

Resolución CREG 101 001 de 2022: la ingesta que diseñan hoy tiene que sobrevivir a esto

75 %

de usuarios con medición avanzada,
meta 2030

< 8:00 a. m.

los OR publican cada día los datos
registrados

> 1.000 kWh/mes

usuarios priorizados en la primera fase

× 96

valores por día con medida de 15
minutos, frente a 1 diario hoy



Hagan la cuenta

Hoy: 355 series × 1 valor/día. Con AMI: millones de medidores × 96 valores/día. Un notebook que lee un Excel no sobrevive; una carpeta vigilada con esquema explícito y checkpoint, sí.



Qué cambia en el diseño

Más archivos, más pequeños y más frecuentes → notificaciones de archivo en vez de listar; particionar por día de dato; la republicación deja de ser mensual y pasa a ser continua (lecturas estimadas → reales).

Fuente: gestornormativo.creg.gov.co/gestor/entorno/docs/resolucion_creg_101-1_2022.htm

Caso: Octopus Energy y la liquidación semihoraria (MHHS)

Reino Unido: de una lectura mensual a 48 valores diarios por cada uno de 8 millones de clientes

× 48

más datos por cliente al pasar a liquidación semihoraria

25 000 M → 300 M

filas procesadas por corrida (−98,8 %)

US\$ 23,63 → 0,48

costo por fecha de liquidación (≈ 50×)

semanal → diaria

frescura del margen por cliente

Qué hicieron	Por qué importa para nosotros
Change Data Feed de Delta para procesar solo lo que cambió	Saber qué cambió exige metadatos de ingesta: <code>_publication_date</code> , <code>_source_file</code>
Tres flujos independientes (liquidación, tarifa inteligente, tarifa estándar), cada uno con su frecuencia	No todo necesita la misma frescura: bronce por llegada, oro diario
Particionado por tiempo y ubicación; serverless para comparar variantes lado a lado	Lo mismo que hacen hoy en Free Edition: serverless, iterar rápido

Tres ingenieros. US\$ 1 M al año evitado. La arquitectura no era el problema; el patrón de procesamiento sí.

Fuente: databricks.com/blog/scaling-mhhs-how-octopus-energy-achieved-50x-cost-reduction-margin-data-engineering

Más casos del sector: ingesta a escala

Casos publicados por el proveedor; los números son verificables en los enlaces de lecturas.md

Empresa	Qué ingieren	Cómo	Resultado
Shell	> 3 millones de sensores; > 5 billones (10^{12}) de mediciones acumuladas	Plataforma de series de tiempo sobre Delta, liberada como código abierto (RTDIP); liquid clustering por sensor y tiempo	Consumo en tiempo real $\times 15$ en cinco años; 25 000 descargas/mes del SDK
SSE Airtricity (Irlanda)	Miles de millones de filas de medidores inteligentes; 48 lecturas/día por medidor	Ingesta y transformación nocturna con pipelines declarativos (clase 5); Unity Catalog	Recomendaciones de ahorro a 65 000 clientes; 4 meses de idea a producción
UK Power Networks	Alertas de medidores inteligentes en tiempo real (cortes, voltaje); carga de red	Unity Catalog + Delta Sharing entre equipos; modelos de carga	£45,7 M de valor en 3 años; £1 M/año por menor tiempo de interrupción
Southern Company (EE. UU.)	4,6 millones de medidores AMI (Alabama, Georgia, Mississippi Power)	Ingesta de telemetría, AMI y SCADA sobre la plataforma	Modelos de cortes y tormentas sobre datos de medidores

Fuente: databricks.com/blog/developing-time-series-lakehouse-shell · databricks.com/customers/sse-airtricity · databricks.com/customers/uk-power-networks · databricks.com/blog/unlocking-future-energy-smart-meter-innovation

Free Edition vs. Azure Databricks: qué simulamos hoy

El código de Auto Loader es el mismo: cambia la ruta y cambia el disparo

Elemento	Hoy en Free Edition	En XM sobre Azure
Carpeta vigilada (landing)	Volumen de Unity Catalog	Contenedor ADLS Gen2 como external location
Detección de archivos nuevos	Listado del directorio	Notificaciones de Event Grid (cloudFiles.useNotifications)
Disparo	Manual, trigger availableNow	Job con file arrival trigger, o programado cada hora
Llegada del archivo	Ustedes lo copian al volumen	Un job descarga del portal de XM / API SIMEM y lo deja en landing
Fuente de base de datos	No aplica	Lakeflow Connect (CDC) sobre la base del SIC
Checkpoint	En el volumen	En ADLS, respaldado; nunca en almacenamiento efímero

Lab 4 (1/2): 23 publicaciones, un proceso que las carga

clases/c04-ingesta/notebooks/lab04_bronce_auto_loader · widget usuario

1

Excel → 23 CSV en landing

Un archivo por FechaPublicacion. Simula 23 llegadas

2

Auto Loader → bronce (falla a propósito)

ingestar_bronce() llama a funciones que aún no existen. Léela: es todo el pipeline

3

Implementar en src

resolver_alias() y con_metadatos() en xm_demanda/ingest/bronze.py; celda de prueba

4

Ingesta y correr otra vez

145.408 filas, 23 archivos, las dos veces. Esa es la idempotencia

```
lab04 · paso 4

# paso 4
ingestar_bronce(landing, tabla, chk)

SELECT count(*),
       count(DISTINCT _source_file)
FROM demanda_raw_bronze;
-- 145408 · 23

# otra vez
ingestar_bronce(landing, tabla, chk)
-- 145408 · 23
```

Si la segunda corrida duplica, algo tocó el checkpoint. Revisen la ruta antes de seguir.

Lab 4 (2/2): llega raw_v2 y hay que decidir

7 días ya cargados, valores corregidos, columna Valor renombrada a ValorKwh

5 Llega raw_v2

Correr la ingesta. Mirar `_rescued_data`: ahí viajó ValorKwh

6 Reconstruir desde landing

DROP TABLE + borrar checkpoint + correr. 150.350 filas, 24 archivos, 0 nulos

7 Vista vigente

row_number por serie-día; 145.408 filas; 7 días con dos versiones

8 ADR-002

Tres decisiones, tres alternativas descartadas, estado aceptada

9 verificar() · commit · PR

feat(c04): bronce idempotente con Auto Loader

Criterios de aceptación

- Bronce con 150.350 filas y 24 archivos, sin Valor nulo
- `_ingested_at`, `_source_file`, `_publication_date` presentes
- Vista vigente con 145.408 filas
- ADR-002 aceptada; test de bronce.py en verde
- `verificar()` en OK; PR en verde

Hito 2 cerrado

Comparar con s04:
`github.com/<docente>/Databricks-to-XM/compare/s04...<tú>:Databricks-to-XM:main` · o `git diff s04 -- src/xm_demanda/ingest docs/adr`

Así sería en Azure Databricks: landing en ADLS, disparo por llegada

Hoy	En XM
Copian raw_v2 al volumen a mano	Job programado descarga del portal XM / API SIMEM y deja el archivo en abfss://landing@< cuenta>.dfs.core.windows.net/demanda/
Auto Loader lista el directorio	cloudFiles.useNotifications = true: Event Grid + cola; escala a millones de archivos
Trigger availableNow ejecutado por ustedes	Job con file arrival trigger sobre la external location; reintentos y alertas
Checkpoint en el volumen	Checkpoint en ADLS con soft delete; incluido en el runbook de recuperación
Republicación simulada (raw_v2)	Llega sola: XM publica ajustes; el job los descarga; bronce acumula; la vista elige
Base del SIC: no aplica	Lakeflow Connect con CDC hacia bronze_energia, mismo esquema de metadatos

Lo que no cambia: esquema explícito, rescate, checkpoint, tres metadatos, vista vigente, ADR-002.

Resumen de la clase 4 y preparación para la clase 5

Logros de hoy

✓ El dato llega tarde y se corrige

Versiones de liquidación: bronce acumula, la vista elige

✓ Auto Loader con esquema explícito

cloudFiles, rescate, availableNow

✓ Checkpoint = estado

Correr dos veces no duplica

✓ Tres metadatos

_ingested_at, _source_file, _publication_date

✓ Hito 2 cerrado

Bronce idempotente con historial

Próxima clase

➔ Calidad y pipelines declarativos

Expectations warn / drop / fail; tabla de cuarentena

➔ Reglas de negocio

valor ≥ 0 , pérdidas $\leq$ demanda, regulado $\Rightarrow$ sin clasificar

➔ De bronce a plata

Lakeflow Declarative Pipelines lee de TU bronce

➔ Traer a la clase 5

Lab 4 terminado: plata no existe sin bronce



Gracias

Nos vemos en la clase 5: Calidad de datos y pipelines declarativos

DOCENTE

Manuela Larrea Gómez